

Architecture over Parameters: How Far Agentic Design Patterns Carry a Weak Model in Automated Machine Learning

Stanislav Chumakov^a, Nikolay O. Nikitin^a

^a*ITMO University, Saint Petersburg, Russia*

Abstract

Large language models have made automated data science agents practical, but the field still lacks a measurement of what the surrounding architecture is worth. Reported gains routinely confound the choice of backbone model with the choice of harness, and agentic systems are seldom compared against baselines that a competent practitioner would tune. We hold the model fixed at a single open 31B-parameter model and vary the architecture across three systems that occupy distinct points of the design space: a rigid pipeline that configures a fixed AutoML backend, a six-agent multi-agent system, and an open agentic harness. All are evaluated inside a common execution environment on the official temporal splits of a real-world tabular benchmark, against eighteen expert-tuned reference methods.

Three findings follow. First, architecture is worth about half of the achievable range: normalised against the published baselines, the systems span 0.26 to 0.84 under an identical backbone as they ship, and 0.26 to 0.75 once the prescribed-knowledge layer is removed from the system that carries one. The strongest of them is level with tuned LightGBM at 0.85 and ahead of tuned CatBoost at 0.80, but does not reach tuned XGBoost at 0.87 across the eight tasks. Second, the relationship is not monotonic in architectural weight. The rigid pipeline ranks at the bottom of the leaderboard, the multi-agent system ranks highest, and the open harness falls between them. Separating the strands changes how much of that ordering is architectural: with its prescribed-knowledge layer removed the multi-agent system falls from 0.84 to 0.75, still ahead of the open harness at 0.72 but by 0.03 rather than 0.13. The ordering survives; most of the margin between the two agentic systems was prescription rather than architecture. Third, the value of prescriptive domain knowledge changes sign with the starting point: it helps where a weak model would otherwise reach for an unsuitable primitive, and hurts where a working baseline already encodes the right choice. The two directions are not of comparable weight. Measured at three attempts per cell, the cost is total on three tasks of eight—the treated branch exhausts its budget and submits nothing at all—while a defensible benefit appears on one. We further show that what the literature treats as one “knowledge injection” intervention is at least two, prescribing which tool to reach for and prescribing how to train and validate; that the tool prescription is the smaller part of the layer it names; and that reading the traces of the unprescribed system explains the boundary directly, since that system selects the prescribed family of tool unaided on every tabular task and never searches its hyperparameters. This offers an account of why aggregate ablations of knowledge injection have reported inconsistent effects.

Keywords: automated machine learning, LLM agents, agentic design patterns, tabular learning, agent scaffolding, evaluation

1. Introduction

Automated machine learning has been reshaped by large language models. Systems built on them plan experiments, write and repair code, read execution logs, and interpret their own results, capabilities that classical AutoML frameworks, which search a fixed space of pipelines, never had. The design space of such systems is now large and largely uncharted: implementations differ in how much autonomy they grant the model, how they inject knowledge about libraries and methods, and how tightly they constrain the sequence of actions.

What has not been measured is how much any of this is worth. Papers in this area typically introduce one system and report that it improves on a previous one, with the backbone model, the scaffold, and the evaluation protocol all changing at once. Two consequences follow. Comparisons across papers cannot separate the contribution of the architecture from that of the model, and, more uncomfortably, agentic systems are rarely

measured against the baseline a competent practitioner would reach for. Where a tuned gradient boosting model appears at all, it usually appears untuned.

This paper isolates the architectural axis. We fix the backbone to a single open 31B-parameter model and vary only the surrounding system across three points of the design space:

- **FEDOT.LLM**, a rigid pipeline in which the model interprets the task and configures a fixed AutoML backend, with code generated from templates that carry fixed input/output contracts;
- **AutoDS-Tools**, a multi-agent system of six specialised agents communicating through structured reports, with a shared toolkit and broad freedom of action;
- **Terminus-2**, an open harness that gives the model a shell, a working directory and an execution loop, and prescribes nothing else.

All three are executed inside a common environment that records every attempt, error, token and unit of cost, and all are evaluated on the official temporal splits of TabReD [1], a benchmark of real-world tabular tasks with distribution shift, against the eighteen expert-tuned reference methods published with it. Because those references are Optuna-tuned and averaged over fifteen seeds, they constitute a ceiling that reflects competent practice rather than a default configuration.

Contributions.

1. **A measurement of the ceiling.** Under an identical backbone, the architectures span 0.26 to 0.84 of the range defined by the published baselines, while tuned XGBoost sits at 0.87 and the best published method at 0.95. The strongest architecture we tested is level with tuned LightGBM at 0.85 and ahead of tuned CatBoost at 0.80, but does not close the gap to XGBoost, from which it is 0.02 short. We state this as a boundary rather than a defeat: it delimits what scaffolding can and cannot buy at a given model scale.
2. **A non-monotonic relationship, and a mechanism for it.** Performance does not increase with architectural weight, and it does not decrease with it either. The most constrained system ranks near the bottom of the leaderboard, and the highest-ranking of the three is neither extreme. We argue that constraint pays only where it removes a decision the weak model reliably gets wrong, and costs elsewhere, and we give direct evidence from an ablation whose sign flips between task families—though the two directions carry very unequal weight, the cost being total on three tasks of eight and the benefit defensible on one. Both arms that separate architecture from prescription and provisioning have now been run: provisioning is worth nothing, prescription is worth three times the architectural gap between the two agentic systems, and the ordering survives both (Section 6).
3. **A separation of two axes usually conflated.** What the literature treats as one “knowledge injection” intervention is at least two: prescribing which class of tool to use (K_{tool}) and prescribing how to train and validate (K_{disc}). We show by direct measurement of the instruction layer that the tool prescription is between 9.1% and 26.0% of the text it gives its name to, that the two halves are strongly sub-additive rather than complementary—the tool half alone outscores both halves together—and, by reading the traces of the system that receives no prescription, that on tabular data the tool half prescribes what the model already chooses unaided. Together these offer an explanation for the inconsistent effects reported for aggregate knowledge and memory ablations.

Relation to the neutral-scaffold assumption. Terminal-Bench 2.0 introduces Terminus 2 as a deliberately minimal scaffold, a single tool and a headless terminal, built to mitigate the biases of richer agent scaffolds so that a leaderboard over it reads as a comparison between *models* [2]. That reading is only sound if the scaffold contributes little. It is already under pressure in

the coding domain, where holding the model fixed and swapping the harness moves aggregate score by more than twenty points [3] and tokens per solved task by up to a factor of forty [4]. We ask the question where a strong non-agentic reference exists and can calibrate the answer: with the backbone held constant, the three systems as they ship span 0.59 of the range defined by eighteen expert-tuned baselines, and 0.49 of it once the prescribed-knowledge layer is removed from the system that carries one, so that what remains is architecture alone. Holding the architecture fixed and swapping the backbone instead, across an elevenfold spread in price, moves it by at most 0.24 — and that larger figure rests on a single task, without which it is 0.07. A scaffold is never neutral; on this domain it is the dominant term of the two.

2. Related work

Degrees of autonomy, and where knowledge enters.. Existing systems can be ordered by how much of the modelling decision they leave to the model: hyperparameter optimisers, where language models mainly address the cold start; pipeline composers searching jointly over features, model and hyperparameters; and adaptive planners that revise the search plan as validation feedback arrives. Orthogonally they differ in where knowledge of libraries and methods comes from. Some rely on the parametric memory of the backbone; some retrieve auxiliary material, for instance strategic insights mined from competition write-ups [5]; others embed API-level knowledge in the decision process, as MLZero does by combining semantic and episodic memory to suppress hallucinated and outdated API calls [6]. Section 4 argues that this second axis conflates two interventions with different mechanisms.

Ablations that disagree.. Multi-agent frameworks for competition-style data science are numerous [7, 8, 5, 6], and their reported ablations disagree instructively. On MLE-bench, search structure is the largest single contributor in one system and has no measurable effect in another, with memory ranked last in one study and level with search in a second [9]. That is expected when ablations are run one factor at a time on different backbones and task sets, and the factors are also shown to interact: advanced search policies gain nothing when restricted to a weaker operator set [9].

Evaluation against tuned baselines.. Agentic AutoML papers that compare against classical methods almost always compare against defaults. The closest work in domain is GRACE-DS [10], which evaluates LLM-powered AutoML agents on tabular tasks under a structured environment and reports a “capability equalizer” effect: structuring the interaction compresses the spread across backbones. Two differences matter. Its normalised scale is anchored on an *untuned* HistGradientBoosting oracle, so its unit of “solved” sits well below the tuned references we use; and its eight backbones are frontier and mixture-of-experts models in the 80B–1.6T range, with no dense small model evaluated. Its results point the same way ours do: the largest gains from structure accrue to the strongest backbones.

Scaffolding against model capability. Whether scaffolding can substitute for capability has been asked directly in the embodied setting, where reasoning, memory, reflection and reinforcement modules were crossed over a capability ladder of dense models, concluding that external scaffolding can compensate for weaker native long-horizon reasoning [11]. We take that design as precedent and ask the same question where a strong non-agentic reference exists, which supervised tabular learning provides and embodied benchmarks do not. Agentic coding has produced parallel evidence that the harness is a large hidden variable: across configurable harnesses evaluated identically, aggregate score spans 23.8 points [3], and harness choice induces up to a fortyfold difference in tokens per solved task while the model-to-model pass-rate spread stays within eight points [4]. That literature varies the harness with the task family fixed; we vary the architectural weight of the system with the family fixed *and* a tuned non-agentic reference available on the same split, which turns a spread into a calibrated fraction of an achievable range. A third result bears on our design: when a coding harness is evolved automatically, ablations localise the gain to tools, middleware and long-term memory rather than to the system prompt [12]. Our prescribed-knowledge axis is a prompt-level intervention, which is why Section 4 treats it as an object of measurement rather than a design pattern, and our finding that its two halves are sub-additive is what that localisation predicts.

3. Three architectures as design points

We describe each system by the questions that matter for our argument: where a modelling decision is taken, who can override it, how many execution iterations are available, and what is fixed beyond the model’s reach.

3.1. FEDOT.LLM: a rigid pipeline

FEDOT.LLM pairs an LLM-driven agent with the FEDOT framework [13] for tabular tasks, in four stages: data understanding extracts paths, types, target, metric and submission format; configuration and code generation produce a task-matched FEDOT configuration and select a template with fixed input/output contracts, leaving only the implementation to the model, with helper functions encapsulating correct API usage; an execution and correction loop runs the code in a sandbox, patching and re-running until success or an iteration cap; reporting persists the script, the trained pipeline and the predictions. The decisive property here is that the choice of modelling tool is not a decision at all but the architecture: the system cannot solve a task with anything other than FEDOT, and accepts only tabular inputs.

3.2. AutoDS-Tools: a multi-agent system

AutoDS-Tools coordinates six specialised agents communicating through structured reports: an Analyst performs exploratory analysis, a Researcher studies the target library and produces usage cookbooks, a Planner turns both into an execution plan, a Coder refines the solution against execution results, a Debugger absorbs execution errors without disrupting

Table 1: The three systems as points in the design space. The last three rows give their position on the prescribed-knowledge axis of Section 4.

	FEDOT.LLM	AutoDS-Tools	Terminus-2
Tool choice	fixed by design	free	free
Role decomposition	none	six agents	none
Plan	fixed stages	generated	emergent
Execution loop	bounded retries	per-agent	open
Input modalities	tabular only	unrestricted	unrestricted
K_{tool}	<i>welded in</i>	switchable	switchable, off
K_{disc}	switchable	switchable	switchable
Left to the model	configuration	everything	everything

the main context, and a Presenter audits the result. The chain is literal rather than metaphorical—each stage writes a Markdown report that the next reads, so a trial directory contains `analyst_report.md`, `researcher_report.md` and `planner_report.md` as artefacts. Freedom of action is broad: any library in the environment, and restructuring between iterations.

3.3. Terminus-2: an open harness

Terminus-2 provides a shell, a working directory and an execution loop, and prescribes nothing about how the task should be solved. There is no planner, no role decomposition and no template. The model reads the task statement, inspects the data, writes code, runs it, reads the output, and repeats until it emits a submission or exhausts its budget.

4. The prescribed-knowledge axis

Interventions that supply a system with knowledge it would otherwise have to derive are usually treated as one thing. In our material they are two, they are of very unequal size, and they do not add.

K_{tool} — tool prescription. Which class of instrument to reach for: a dedicated tabular AutoML library for tabular data, a vision backbone library for images, a graph learning library for graphs, a transformer library for text. It removes the question “what does one normally use for this?”

K_{disc} — training discipline. How to train and how to check oneself: the validation protocol, the handling of temporal shift, leakage checks, and the prohibition of degenerate submissions. It is orthogonal to the choice of library.

Separating them is not a matter of tidiness: crossed properly, the two halves do not add. On the normalised scale of Section 6 the tool half alone reaches 0.87, the discipline half alone 0.77, and the two together 0.85—the stronger half does better on its own than in company. The interaction is sharper still on the second architecture, where the library block alone loses 19 of 22 trials to the time limit while adding the discipline block on top brings that to 11 of 24: there the half that harms alone protects in combination (Section 7.4).

`amp-parkinsons` is a cautionary case about doing this by inference instead. An earlier draft attributed a degradation there

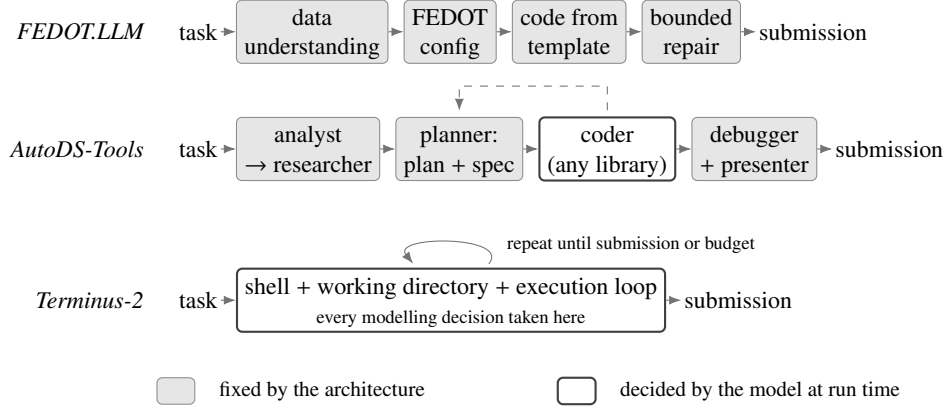


Figure 1: The three systems on one schematic. Shaded stages are fixed by the architecture and cannot be renegotiated by the backbone; unshaded stages are where the model actually decides what to do. The amount of prescribed structure falls from top to bottom, and—as Section 6 shows—measured performance rises in the same direction.

to a prohibition of constant submissions said to have been added for that task alone. Fingerprinting the layer text shipped with each of the ten MLAGentBench tasks shows otherwise: the prohibition belongs to the discipline block and reaches every task of a family identically, so what differs is not the task but the edition of the layer (Section 11). The figure that draft compared against came from the other edition and cannot be re-read from any artefact, so we drop it. Repeating both branches three times gives numbers that can: the layer as a whole moves `amp-parkinsons` from a mean SMAPE of 62.28 without it to 86.46 with it, and each branch is tight enough about its own mean—58.21 to 66.35 without, 81.78 to 90.08 with—that the direction is not in doubt. While the two halves are delivered as a single instruction layer, no row of the ablation can be attributed to either, and the tables report the layer as a whole.

4.1. The layer is named after its smallest part

In the implementation we study the layer is composed literally as `instruction + LIBRARY[family] + DISCIPLINE`, and the first section of `DISCIPLINE` is a `HARDWARE` block that prescribes neither a tool nor a training protocol. That makes the split measurable rather than conceptual. Reading the character counts from the module that composed the layer, and checking them against the instruction each container received: the `LIBRARY` block runs from 369 characters for text to 1291 for tabular tasks, the `HARDWARE` block is a fixed 668 and the `DISCIPLINE` block a fixed 3008. The tool prescription—the intervention the layer is named after—accounts for between 9.1% and 26.0% of the text. The `HARDWARE` block, pinning down how the device is resolved before training, is neither tool prescription nor training discipline and forms a third category; the `DISCIPLINE` block is the largest component on every task. On the tabular tasks the whole layer runs to 4972 characters against a task statement of 1050, close to five times the length of the problem it is attached to. This matters for attribution: any effect ascribed to tool prescription in an aggregate ablation is at least as likely to originate in the two fixed blocks travelling with it, and those are constant across modalities while the reported effects are not.

4.2. A prescription presupposes its environment

Tool prescription is realisable only where the prescribed tool exists, which makes it a property of the environment as much as of the text. The layer states that the specialized library is “pre-installed” and forbids falling back on general-purpose estimators, so it presumes an image provisioned accordingly. Against an image without that library it does not produce a weaker treatment but a failure. We met this directly: the benchmark’s own image ships `numpy`, `pandas` and `scikit-learn` on Python 3.13, on which the prescribed tabular library does not build, so an agent given the prescription there reads the data, writes a correct pipeline, and fails at import, scoring zero while having done nothing wrong. The environment is therefore part of the treatment and we report the provisioning of each arm explicitly (Section 5). This is also a caution for the literature: an ablation of “prescribe a specialized library” measures the prescription only where the provisioning already holds, and otherwise measures the provisioning.

4.3. Where each system sits

Table 1 places the three systems on this axis.

FEDOT.LLM is not missing K_{tool} but has it welded into its architecture: the instruction “solve this through FEDOT” is the pipeline. That is the extreme point of the design rather than a hole in it, and it is what makes Section 6 interesting—the system whose tool prescription is maximal and non-removable performs worst on exactly the family the prescribed tool was built for. Only K_{disc} can therefore be crossed with every system; K_{tool} is crossed on AutoDS-Tools and Terminus-2, with FEDOT.LLM entering as the reference point at which it cannot be switched off.

5. Experimental design

5.1. Factors

- **Architecture:** FEDOT.LLM, AutoDS-Tools, Terminus-2.

- K_{tool} : on/off, crossed on AutoDS-Tools and Terminus-2 (Sections 7.1 and 7.4). Terminus-2 has no prescription of its own to switch, so the text is appended to the task file instead; provisioning, the nearest thing to a library manipulation that does not touch the instruction, is varied separately in Section 6.1.
- K_{disc} : on/off, crossed on AutoDS-Tools and Terminus-2 (Sections 7.1 and 7.4).
- **Backbone model**: gemma-4-31b-it throughout, with gemma-4-26b-a4b-it below it and glm-4.7 above it, crossed on AutoDS-Tools and Terminus-2 (Section 7.5). The upper point leaves the family because that family has nothing larger, so this axis varies backbone capability and not parameter count.
- **Task protocol**: greenfield (nothing supplied) versus improve-a-baseline (a working training script supplied).

5.2. Benchmarks

TabReD. Eight real-world tabular tasks with temporal splits [1]. The benchmark ships no agentic protocol, so tasks are solved from scratch: this is our greenfield condition. Its value here is the published reference table of eighteen methods, spanning gradient boosting implementations, tabular deep learning architectures, ensembles and retrieval-augmented models, tuned with Optuna on the official validation split and averaged over fifteen seeds. Metrics are ROC-AUC for classification and RMSE in raw target units for regression.

MLAgentBench. Ten tasks spanning tabular, text, vision and graph data [14]. Each supplies a working but weak `train.py` and asks the agent to improve it: this is our improve-a-baseline condition. FEDOT.LLM participates only in the two tabular tasks, for the structural reason given in Section 9.

This benchmark carries no reference table. Unlike TabReD it ships no tuned external methods, and none exist in the literature at the granularity we would need, so there is no range against which a score can be normalised and no external ceiling to fall short of. Every quantity we report on MLAgentBench is therefore a within-study contrast: one branch of a system against another, one architecture against another, or one edition of an instruction layer against another, always under conditions we held fixed ourselves. The normalised 0-to-1 language of Section 6 belongs to TabReD alone and should not be carried across. What MLAgentBench contributes is the second starting point—a working baseline to improve rather than an empty directory—and the signs and failures that condition produces.

Scientific case studies. Three published studies with author-reported baselines, used in Section 10 to test external validity outside benchmark conditions. Their instructions carry the heaviest prescription of any condition in this paper: library, featurisation, leakage handling and the baseline to beat. They are read as the maximal-prescription end of the axis rather than as an unaided-agent condition.

5.3. Common execution environment

All runs execute inside the Harbor framework, which records for every trial the submission, the scorer output, the number of attempts and errors, wall-clock time, token counts and monetary cost. Reporting cost and attempts alongside accuracy lets us treat efficiency as a second axis rather than an afterthought.

5.4. Provisioning

Because a tool prescription presupposes the tool (Section 4), we state the container contents per arm rather than assuming a neutral environment. The Terminus-2 and FEDOT.LLM runs reported here used the benchmark’s stock image: `numpy`, `pandas` and `scikit-learn` on Python 3.13. The AutoDS-Tools arm also requires the libraries its prescription names, on Python 3.12, which is the environment its earlier runs also had. Data, splits, metrics and the backbone model are identical across arms; the installed library set is not, and we report it as a factor.

The same honesty is owed about the instruction. Each system enters the leaderboard in the configuration it is shipped and used in, which is not the same configuration for all three: Terminus-2 receives the task statement alone, FEDOT.LLM receives it through a pipeline whose tool choice is welded in, and AutoDS-Tools receives it with its prescribed-knowledge layer applied, that layer being the system’s default. A row of Table 2 therefore compares deployed systems rather than isolated architectures. Two further arms separate the strands, AutoDS-Tools with the layer disabled on the common adapter and Terminus-2 on the enriched image, and each is a single sweep of the same eight tasks. We ran both, and report them in Sections 7.1 and 6.1. The first arm was run as a full 2×2 rather than as a single off-switch, since the layer has two halves that can be disabled independently.

5.5. Noise floor

Rather than assume a significance threshold, we measure one. Terminus-2 and AutoDS-Tools were each run three times on every TabReD task; The two systems do not share a floor. Terminus-2’s relative standard deviation ranges from 0.03% to 0.94%; AutoDS-Tools reproduces itself an order of magnitude more tightly, from 0.00% to 0.10%. On six of the eight tasks for Terminus-2 and three of the eight for AutoDS-Tools, two attempts are bit-identical, meaning the agent reproduced the same script, so the repeats carry less independent information than their count suggests.

We therefore treat relative differences above 1% as real when comparing the two systems, which takes the threshold from the noisier of the pair and is the conservative choice there. That threshold would be far too coarse for comparing configurations of one system against each other: for AutoDS-Tools the appropriate figure is set by its worst task, `maps-routing` at 0.07%, while three of its eight tasks reproduce to every digit across all three attempts. We use 0.1% for those comparisons. One caveat applies to both figures. Attempts that agree to every printed digit, of which there are several, are not independent samples of the agent: on these tasks the submission is a deterministic

library call, so any two attempts that reach the same call return the same number (Section 7.4). The floors below therefore measure library determinism as much as agent stability, which makes them optimistic in exactly the way a noise floor should not be. The MLAGentBench cells are no longer single runs. We repeated them at three attempts per cell—sixty trials across the two branches of the layer ablation, twenty-four more for the second architecture, and twenty-four more for the second edition of the layer (Sections 7, 9 and 11). Two tasks resisted repetition on hardware without an accelerator and keep their single-run figures (Section 9).

The repeats supply the noise floor this benchmark previously lacked, and it is nothing like the tabular one. Where the TabReD floor is a fraction of a percent, three attempts of one configuration on ogbn-arxiv span 0.2237 to 0.4790 and on fathomnet 0.0844 to 0.3846; the second architecture returned 0.1144, 0.3303 and 0.5569 on fathomnet in an earlier job and 0.0844, 0.2629 and 0.3846 in a later one. A difference on this benchmark is worth reading only when it exceeds a spread of that size, which is why Section 7 rests on cells that go to zero rather than on cells that move a little.

Repetition also changed how we count a lost trial, and the change matters beyond this paper. On TabReD a trial that reaches the task ceiling has written nothing, so excluding it loses no information. On MLAGentBench that inference is false: the verifier scores whatever `submission.csv` exists at the moment the agent is stopped, and trials that exhausted their budget frequently submitted usable work—`clrs` scored 0.4118 and 0.4404 on two such attempts and `feedback` 0.6492 on a third. Exhausting the budget and returning nothing are separate events here, and we count them separately throughout.

5.6. What is not crossed

Two limitations of the design are deliberate. FEDOT.LLM cannot enter the non-tabular cells at all. And the AutoDS-Tools TabReD runs reported in earlier drafts were computed on a different data slice; they are excluded from Table 2, which uses a re-run on the common adapter. Both sets are released, because the difference between them is itself the measured price of the data preparation.

6. Where architecture stops

Table 2 places the three systems that ran on the official TabReD splits into the benchmark’s own reference table, as if they were methods. Table 3 restates the same runs on a per-task normalised scale, where 0 is the weakest and 1 the strongest of the eighteen published baselines.

Architecture is worth about half the range.. With the backbone held fixed, the normalised mean moves from 0.26 for the rigid pipeline to 0.85 for the multi-agent system as deployed, a spread of 0.59, with the open harness at 0.72 between them; in average rank, 18.3 against 7.0 among twenty-one methods, with the open harness at 11.4. Part of that spread is prescription rather than architecture: the multi-agent system without its

instruction layer scores 0.75, so the span attributable to architecture alone is 0.49. We use the smaller figure whenever the claim is about architecture and the larger only where the systems are compared as they ship.

But it does not reach competent practice.. Tuned XGBoost sits at 0.87 (rank 5.0) and the strongest published method at 0.95 (rank 4.1). The best of our systems reaches 0.85 at rank 7.0, which places it inside the gradient-boosting band rather than below it: LightGBM is at 0.85 and CatBoost at 0.80, so the multi-agent system is level with the first, ahead of the second, and short of XGBoost by 0.02. The honest statement is a narrower boundary than the usual one: *with an open 31B model, the best architecture we tested matches the middle of competently tuned gradient boosting on real-world tabular tasks under temporal shift, but not its best member, and no architecture we tested approaches the strongest published method.* We state it plainly because the comparison is usually made against defaults, where the same systems would appear to win outright.

The ordering is not monotonic in either direction.. The system that ranks highest is neither the most nor the least constrained. Read as it stands the relationship is an inverted U—rigid pipeline 0.26, multi-agent 0.85, open harness 0.72—but each system enters in the configuration it ships in, and those differ in more than architecture: the multi-agent row carries its prescribed-knowledge layer and an image provisioned with the libraries that layer names, while the open harness receives the task statement alone in the stock image. Two arms separate those strands, and both have now been run. Provisioning alone does nothing (Section 6.1); prescription does almost everything. With its layer removed—the *neither* column of Table 6, same tasks, same image—the multi-agent system scores 0.75 against 0.85 as deployed, with the open harness at 0.72. The ordering survives the separation but the distance does not: 0.13 as deployed becomes 0.03 once neither system carries a prescription, so on these eight tasks the layer is worth roughly three times the architectural difference between the two. Neither the comparison with the rigid pipeline at 0.26 nor the gap to XGBoost is affected.

6.1. Provisioning alone does nothing

We ran the open harness on the image built for the multi-agent system, same eight tasks, three attempts each. Seven of the eight land inside this system’s own noise floor—relative differences from +0.08% to −0.93%, with four of them at or below 0.08% and several reproducing the stock-image attempts to every digit printed. The reason is in the traces rather than the metrics: across all twenty-four trials the agent named none of the three libraries the enriched image pre-installs. It went to gradient boosting by hand, as on the stock image, where the library was already reachable because it had installed it itself. Provisioning without a text that points at it changes the container and not the run.

That the enriched image was itself defective makes the point sharper. It installed the prescribed libraries on a base without the OpenMP runtime they link against, so LightGBM imported

Table 2: Agent architectures placed in the TabReD leaderboard. All rows are computed on the full official temporal splits with the benchmark’s own metrics in raw target units. Published baselines are Optuna-tuned means over 15 seeds [1]; the agent rows use a single backbone model (gemma-4-31b-it). Best per column in bold. A superscript on an average rank gives the number of tasks it was averaged over, where that is fewer than all eight.

Method	ROC-AUC ↑			RMSE ↓					Avg. rank
	Homesite	Ecom	HomeCred	Sberbank	Cooking	Delivery	Maps	Weather	
<i>GBDT</i>									
XGBoost	0.9601	0.5763	0.8670	0.2419	0.4823	0.5468	0.1616	1.4671	5.1
LightGBM	0.9603	0.5758	0.8664	0.2468	0.4826	0.5468	0.1618	1.4625	6.4
CatBoost	0.9606	0.5596	0.8621	0.2482	0.4823	0.5465	0.1619	1.4688	6.9
RandomForest	0.9570	0.5764	0.8269	0.2640	0.4884	0.5959	0.1653	1.5838	17.0
Linear	0.9290	0.5665	0.8168	0.2509	0.4882	0.5579	0.1709	1.7679	18.6
<i>Tabular DL</i>									
MLP	0.9500	0.6015	0.8545	0.2508	0.4820	0.5504	0.1622	1.5470	9.4
SNN	0.9492	0.5996	0.8551	0.2858	0.4838	0.5544	0.1651	1.5649	15.2
DCNv2	0.9392	0.5955	0.8466	0.2770	0.4842	0.5532	0.1672	1.5782	16.5
ResNet	0.9469	0.5998	0.8493	0.2743	0.4825	0.5527	0.1625	1.5021	11.6
FT-Transformer	0.9622	0.5775	0.8571	0.2440	0.4820	0.5542	0.1625	1.5104	8.8
MLP (PLR)	0.9621	0.5957	0.8568	0.2438	0.4812	0.5527	0.1616	1.5177	6.6
Trompt	0.9588	0.5803	0.8355	0.2509	0.4809	0.5519	0.1624	1.5187	10.5
<i>Ensembles</i>									
MLP ens.	0.9503	0.6019	0.8557	0.2447	0.4815	0.5494	0.1620	1.5186	7.2
MLP-PLR ens.	0.9629	0.5981	0.8585	0.2381	0.4806	0.5518	0.1612	1.4953	4.1
<i>Training methodologies</i>									
MLP aug.	0.9523	0.6011	0.8449	0.2659	0.4832	0.5532	0.1631	1.5193	13.1
MLP aug. rec.	0.9531	0.5960	0.7453	0.2515	0.4834	0.5541	0.1636	1.5160	13.6
<i>Retrieval-augmented</i>									
TabR	0.9487	0.5943	0.8501	0.2820	0.4828	0.5514	0.1639	1.4666	12.6
ModernNCA	0.9514	0.5765	0.8531	0.2593	0.4825	0.5498	0.1625	1.5062	11.4
<i>Agentic AutoML systems (this work)</i>									
Terminus-2 (<i>open harness</i>)	0.9588	0.5585	0.8590	0.2491	0.4837	0.5482	0.1628	1.5247	11.4
AutoDS-Tools (<i>multi-agent</i>)	0.9611	0.5800	0.8633	0.2515	0.4820	0.5474	0.1620	1.4783	6.6
FEDOT.LLM (<i>rigid pipeline</i>)	0.9400	0.5600	0.7700	0.2800	0.4900	0.5500	0.1700	1.5900	18.2

cleanly and then failed inside `fit_predict` as a broken joblib worker pool—a failure that says nothing about OpenMP on its face. Forty-five trials across the two systems met it; forty-five diagnosed it, reasoned from the dead worker pool to the missing `libgomp.so.1`, installed it and continued. None abandoned the library and none reported the repair. The defect is ours and is recorded as a threat (Section 11); the behaviour is what belongs here. Provisioning is inert as a treatment not because these agents ignore the environment but because they rebuild it, and a system that will install what it needs cannot be studied by installing it in advance.

The one exception is sberbank-housing, at -3.49% with a spread of 0.017 against 0.000 to 0.001 elsewhere: two attempts reproduce the stock values exactly and the third trained a thousand boosting rounds without early stopping, which under temporal shift costs about a tenth of the metric. That is the agent’s own lapse and the subject of Section 7.

The rigid pipeline sits at the bottom of the leaderboard. FE-DOT.LLM’s normalised mean is 0.26 and its average rank 18.3 of 21, below plain linear regression at 0.29 and rank 18.6: the most constrained of the three systems, driving a mature AutoML backend, does not recover the score of the simplest ref-

erence in the table. Its metrics reached us rounded to two decimals, so we checked how much of the placement is an artefact of that rounding: sweeping every task independently to both ends of its interval moves the normalised mean between 0.05 and 0.47 and the rank between 20.3 and 16.6. The ordering against linear regression is therefore *not* resolvable at the precision available to us and we rest nothing on it. What survives the whole interval is what we do use: even at its most favourable corner the rigid pipeline stays 0.25 of the normalised range below the open harness, 0.37 below the multi-agent system, and below every tuned gradient-boosting baseline. We report this about one of our own systems deliberately—it is the lower anchor of the architectural axis, and its position is what gives the axis its length. Full precision would need the original job artefacts of a system we could not rebuild, so we could neither re-run it nor re-score its submissions.

Cost, and what actually binds.. Table 4 reports the Harbor telemetry behind these results.

Token spend is not the binding constraint at this scale: a whole TabReD trial costs the open harness under a cent and the most expensive row is under two. What separates the systems from the tuned baselines is training compute and search, not the

Table 3: Normalised position on each TabReD task: 0 is the weakest and 1 the strongest of the 18 published baselines. The two agent rows share one backbone model, so the spread between them is attributable to architecture alone.

Method	Homesite	Ecom	HomeCred	Sberbank	Cooking	Delivery	Maps	Weather	Mean
Terminus-2	0.88	-0.03	0.93	0.77	0.60	0.97	0.83	0.80	0.72
AutoDS-Tools	0.95	0.48	0.97	0.72	0.82	0.98	0.92	0.95	0.85
FEDOT.LLM	0.32	0.01	0.20	0.12	-0.21	0.93	0.09	0.58	0.26
XGBoost	0.92	0.39	1.00	0.92	0.78	0.99	0.96	0.98	0.87
MLP-PLR ens.	1.00	0.91	0.93	1.00	1.00	0.89	1.00	0.89	0.95
Linear	0.00	0.16	0.59	0.73	0.03	0.77	-0.00	-0.00	0.28

Table 4: Cost and wall-clock per trial, from the Harbor job telemetry. Wall-clock is measured end to end for the whole job and divided by the number of trials and by the job’s own concurrency setting, so it includes environment builds and any queueing; the two AutoDS-Tools rows cover the same eighteen tasks and differ only in the instruction file. Token prices are those of the shared backbone. For the five rows marked *this work* the figure is the mean over the job’s own trials rather than end-to-end job time divided by concurrency.

System	Benchmark	Trials	\$/trial	In tok/trial	Out tok/trial	Min/trial
Terminus-2	TabReD	24	0.0069	49 969	2 111	5.1
Terminus-2	MLAgentBench	6	0.0148	140 094	4 571	11.7
AutoDS-Tools (<i>K</i> on)	both	18	0.0174	140 292	10 017	130.1
AutoDS-Tools (<i>K</i> off)	both	18	0.0122	94 987	7 918	100.2
AutoDS-Tools (this work)	TabReD	24	0.0074	61 606	5 349	44.4
AutoDS-Tools (<i>K</i> off, this work)	MLAgentBench	30	0.0089	76 463	5 864	63.2
AutoDS-Tools (shipped <i>K</i> , this work)	MLAgentBench	30	0.0103	90 439	6 407	70.5
AutoDS-Tools (composed <i>K</i> , this work)	MLAgentBench	24	0.0254	237 421	11 894	48.3
Terminus-2 (this work)	MLAgentBench	24	0.4317	1 989 443	11 873	19.1

Every figure in this table is a mean, and these distributions are strongly right-tailed, so the means sit well above the typical trial. Where the text quotes a median it is doing so deliberately: the median trial of the untreated multi-agent branch on MLAgentBench is 10.9 minutes against a mean of 63.2, because 13 of its 30 trials reached a task ceiling. Means are the right figure for a budget and medians for an expectation; the two are not in conflict.

The Terminus-2 MLAgentBench mean is not a typical trial. One run of twenty-four—*feedback*, which took 571 agent steps and 41.4M input tokens—cost \$8.71 on its own, 84% of the job. The median trial in that job cost \$0.033, cheaper than any AutoDS-Tools trial. The open harness is the cheaper architecture in the middle of its distribution and the more expensive one in its tail, and a per-trial mean hides both facts.

price of the backbone. Nor do the orderings of accuracy and cost agree—the system that ranks highest is also the cheapest per trial, by about 2.5× in tokens and an order of magnitude in wall-clock. The two AutoDS-Tools rows isolate the price of the layer, since they differ only in the instruction file: it raises input tokens by 48%, output tokens by 26% and dollars by 43%, and Section 7 shows that what this buys is positive on some task families and negative on others while the cost is paid on every task.

What binds is the clock. In our re-run, twenty-four trials under the benchmark’s own 3600 s ceiling, agent wall-clock had a median of 38.9 minutes and a minimum of 18.7; four trials finished within fifteen minutes of the ceiling and four more were cut by it. A killed trial and a lost trial are not the same thing: three of the four had already written a submission and are scored like any other, and one had not, so the ceiling truncated the work in one trial of six and destroyed the result in one of twenty-four. The open harness on the same tasks and ceiling has a median of 3.4 minutes and a maximum of 9.3. We report the lost trial as a failure rather than a zero, since a timeout is a property of the budget and not a measurement of the model (Section 11), and note that because truncation falls on the slowest tasks it is not random with respect to the result. An architecture averaging forty-one minutes where another averages four is buying its accuracy partly with a margin it does not

have.

The hour binds rather than the hardware, which a repeat shows. We ran this arm twice, on a twelve-core workstation at concurrency two and on an eighteen-core server at concurrency four: medians 37.4 and 38.9 minutes, so doubling the parallel load and changing the machine moved the distribution by a minute and a half. What moved was the loss rate, two trials of twenty-four against one, because a faster machine leaves more agents with something on disk when the clock stops. A time-out rate is a property of a deployment, not of a system. The wall-clock column should be read accordingly: for the earlier jobs it is end-to-end time divided by trials and concurrency, so it absorbs environment builds and queueing, and the jobs did not share a machine. It is reliable as an order of magnitude and as a within-pair comparison, not as a benchmark of throughput.

7. Why more structure is not better

The ordering in Section 6 does not follow architectural weight: the most constrained system performs worst and the best of the three sits between the extremes. Quantity of structure does not predict outcome. We propose a sharper statement, which the data can test:

Constraint pays exactly where it removes a decision the weak model reliably gets wrong. Every-

where else it removes exploration the model could have recovered from.

Table 5 ablates the prescribed-knowledge layer on AutoDS-Tools over MLAGentBench: both branches run the identical six-agent architecture and differ only in the instruction file. The layer arrives as one block here, so the table isolates the layer rather than K_{tool} alone (Section 4); we split it into halves on TabReD in Section 7.1.

Measured three times per cell, the effect separates the tasks into two groups of very unequal weight. The costly side is unambiguous: on `clrs` the untreated branch scores 0.3909 pointer accuracy over three attempts while the treated branch submits nothing in three, every attempt exhausting the task’s own agent budget; on `fathomnet` the same, 0.1435 against nothing; on `feedback`, MCRMSE 0.8261 with a best attempt of 0.5403, again against nothing. Three tasks of eight lose their entire result to the prescription rather than a fraction of it.

The paying side is thinner than the shape of the split suggests. Only `ogbn-arxiv` gains by a margin clearing the spread of either branch, 0.3866 to 0.5069. `identify-contrails` moves the same way, 0.0022 to 0.0478, but both figures sit near zero on a metric whose useful range does not, so the normalised +0.96 in Table 5 is an artefact of dividing by a small number and we do not read it as a result. `spaceship-titanic` and `house-price` are level within their own spreads, and `amp-parkinsons` moves against the prescription, SMAPE 62.28 without it against 86.46 with it. Prescription is an override; here its cost is large, repeatable and on three tasks total, while its benefit is defensible on one of eight. This benchmark demonstrates the costly half of the mechanism. FEDOT.LLM is the limiting case, permanently in override mode, which is consistent with its position at the bottom of Table 2.

7.1. Splitting the layer

On TabReD the two halves are separately switchable. Table 6 crosses them: four cells, twenty-four trials each, one image, one concurrency setting, one machine, differing in the instruction file and nothing else.

Both halves together beat the untreated cell on all eight tasks, so on this family the layer does no harm. That is also the last thing to survive aggregation. Averaged over the eight tasks the library half is worth +0.64%, the discipline half +0.08%, and their interaction −0.09%: one intervention and one piece of ballast.

Two tasks carry almost all of it, `ecom-offers` moving +3.68% under the library half alone and `homecredit-default` +0.79%, while the other six move by less than half a percent in either direction—at or below what the same cell reproduces across its own three attempts.

The cost columns do not run parallel. Prescribing a library shortens the agent’s search and lengthens its fitting: the median trial goes from 15.4 minutes untreated to 30.4 with the library named, while the bill falls from 0.0086 to 0.0074 dollars. Prescribing discipline does the opposite, adding 11% to the bill and cutting the median trial to 9.7 minutes. Which of these is cheap

depends on which budget is scarce, and Section 6 has already shown that here it is the clock.

7.2. What the unprescribed system chooses on its own

If prescription pays only where the model’s default is wrong, then on a family where the default is already right it should be worth close to nothing. The terminal traces of the open harness test this without an ablation. Its instruction names no library—task, file layout, output format, train and validate any model of your choosing—and it fitted a gradient-boosting model on every task in every attempt: LightGBM in twenty-three of the twenty-four trials of Section 6.1 and XGBoost in the remaining one, and on the stock image it installed LightGBM itself on six of the eight tasks. That is the family the tool prescription would have named, so on tabular data the prescription is not an override but a restatement of the default.

The same traces explain the ceiling. The configuration is hand-written and near-identical across tasks and attempts: 1000 estimators in all twenty-four trials, learning rate 0.05 in twenty-three, early stopping on a holdout in twenty-three, 31 leaves in the nineteen that set a leaf count. No search appears in any trace—no Optuna, no grid, no randomised search, no cross-validated sweep. The gap to the published gradient-boosting rows is therefore not a gap in tool selection, which the harness gets right unaided, but a gap in search: the baselines are Optuna-tuned over fifteen seeds, the agent writes one plausible configuration and stops. Telling a weak model *which* tool to use is worth a great deal where it would otherwise choose badly and nothing on tabular data; what is left unaddressed there—search over the chosen tool—is what no system in our comparison performs. The per-task extraction is released with the paper.

7.3. And which parts of the discipline it skips

Table 6 says the discipline half is worth nothing on the score. It does not say the half does nothing. Read for behaviour instead of accuracy, the same four cells show a treatment landing exactly as written. Three clauses—compare against a trivial predictor, allocate the time budget before starting, keep the best variant by validation rather than the last trained—leave a signature in no trial of either cell where the discipline half is off and in every trial of both cells where it is on. That is as clean a manipulation check as this measurement allows.

Two further clauses separate nothing, and both show why single-arm readings mislead. The submission is inspected in twenty-two of twenty-four trials with no prescription at all: the multi-agent system does this unbidden, and contrasting it with the open harness, which leaves no trace of that clause in any of its twenty-four trials, would credit the text with a habit belonging to the architecture. Early stopping runs the other way, universal without the library and invisible with it, because the prescribed library performs it internally and does not report it. A signature is evidence that a behaviour reached the trace, not proof that it happened.

So the discipline block moves three behaviours from never to always and moves the score by 0.08%. Prescribed method is not inert as instruction; it is inert as intervention, on a family where the agent’s untutored defaults were already adequate.

Table 5: The prescribed-knowledge layer on AutoDS-Tools over MLAGentBench, three attempts per cell. Both branches run the identical six-agent architecture on the identical images and verifier and differ only in the instruction file. On this benchmark the layer arrives as one block, so no row separates K_{tool} from K_{disc} ; Table 6 separates them on TabReD. Brackets give the range over the attempts that produced a submission; *none* marks a cell in which all three attempts exhausted the task’s own agent budget without submitting anything.

Task	Metric	layer off	layer on	Δ (norm.)
identify-contrails	Dice	0.0022 [0.0000–0.0040]	0.0478 [0.0149–0.1137]	+0.96
ogbn-arxiv	Accuracy	0.3866 [0.2237–0.4790]	0.5069 [0.4334–0.5573]	+0.24
feedback	MCRMSE	0.8261 [0.5403–1.2569]	none	—
amp-parkinsons	SMAPE	62.2787 [58.2113–66.3461]	86.4637 [81.7768–90.0803]	-0.28
spaceship-titanic	Accuracy	0.8074 [0.8022–0.8131]	0.8108 [0.8080–0.8131]	+0.00
house-price	MAE	15 894.8 [15 529.7–16 600.8]	16 433.4 [16 162.0–16 904.2]	-0.03
imdb	Accuracy	—	—	—
cifar10	Accuracy	—	—	—
clrs	Ptr. acc.	0.3909 [0.3204–0.4404]	none	—
fathomnet	micro-F1	0.1435 [0.1428–0.1442]	none	—

Table 6: The prescribed-knowledge layer split into its two halves on TabReD. All four cells ran on one image, at one concurrency setting, on one machine, twenty-four trials each; they differ in the instruction file and in nothing else. The first column is the untreated cell in the task’s own units; the next three are relative to it, signed so that positive is better. Almost the whole effect sits in the library half.

Task	Metric	neither (units)	relative to neither, %		
			K_{tool}	K_{disc}	both
homesite-insurance	ROC-AUC	0.9600	+0.10	-0.03	+0.12
ecom-offers	ROC-AUC	0.5649	+3.68	+1.24	+2.67
homecredit-default	ROC-AUC	0.8559	+0.79	+0.20	+0.86
sberbank-housing	RMSE	0.2516	-0.31	-0.00	+0.07
cooking-time	RMSE	0.4835	+0.47	-0.04	+0.30
delivery-eta	RMSE	0.5476	-0.00	-0.05	+0.04
maps-routing	RMSE	0.1627	+0.64	+0.09	+0.45
weather	RMSE	1.4968	+0.14	-0.41	+1.24
mean over tasks			+0.69	+0.12	+0.72
better than neither			6/8	3/8	8/8
median min/trial		15.4	30.4	9.7	38.9
\$ /trial		0.0086	0.0074	0.0095	0.0074

One connection survives with its attribution corrected: the clause the open harness never leaves a trace of is the instruction to inspect the submission before finishing, and the failure recorded in Section 11—that harness submitting one constant label for every row of a 25,000-row test set for exactly 0.5000—is the failure that clause exists to catch. An observed failure and a measured omission are the same fact. Supplying the clause is not sufficient to make it pay: on TabReD the multi-agent system inspects its submission in every trial and scores no better for it, because there was no degenerate submission to catch.

7.4. The same layer in another architecture

Whether either half is worth anything in general is a different question, and the two systems are far enough apart to answer it. The layer is composed on the host and reaches only its own agent, so the only way to hand the same text to the open harness is to put it in the task: we appended the byte-identical halves to each instruction file and ran twenty-four trials per cell on one machine, one image, one backbone, one concurrency setting. Every manipulation check passes—the half that should

be present appears in twenty-four trajectories of twenty-four, the half that should be absent in none.

Accuracy is not where the answer is. In the discipline cell every value lands inside the untreated arm’s own spread, two reproduce an untreated value to every digit, and the mean over the seven tasks that return a number is -0.05% , an order of magnitude inside this system’s 0.94% noise floor.

What moves is whether a trial produces an answer at all. Trials ending without a submission go from two of twenty-four untreated to thirteen with the discipline half and nineteen of twenty-two with the library half, the latter at $p = 9.6 \times 10^{-8}$. Not one is a timeout: every one ends with the agent calling the tool that declares the task finished, which is also how both untreated losses end. The layer introduces no new failure mode; it makes the existing one common.

The failure is one of patience rather than method. The agent launches a fit in the foreground, sees a terminal that has stopped producing output because its own job still holds it, concludes that the reporting is broken rather than that the work is unfinished, and marks the task complete—saying so in as many words in five of the thirteen discipline-cell losses. The trials

Table 7: The same prescribed-knowledge layer, given to the open harness instead of the system it ships with. Five cells of twenty-four trials on one machine, one image and one backbone, differing only in what was appended to the task file. Accuracy is not what moves; whether a trial returns anything at all is. The library half alone loses seven trials in eight, and adding the discipline half on top of it halves that.

Cell	Trials	Lost	Tasks	Steps	Min	\$/trial	rel. %
neither	24	2	8/8	6	2.5	0.0029	ref.
K_{disc}	24	13	7/8	18	14.1	0.0303	-0.05
K_{disc} , minus time	24	0	8/8	7	3.0	0.0044	-0.69
K_{tool}	22	19	3/8	14	10.2	0.0107	+1.21
both	24	11	6/8	20	18.4	0.1172	+1.13

Trials: launched trials that ran to a verdict; the K_{tool} cell lost two to a network fault outside the trial and they are excluded. Lost: ended without writing a submission; no cell had a single timeout. Tasks: how many of the eight returned a metric from at least one attempt. Steps and Min are medians per trial. rel. % is the mean relative change against the untreated cell over the tasks that cell returns, signed so positive is better; the last two rows average over a surviving minority and are not comparable with the first three. Loss counts, Fisher’s exact test, two-sided: K_{disc} against neither $p = 0.0013$; trimmed against K_{disc} $p = 2.6 \times 10^{-5}$; trimmed against neither $p = 0.49$, i.e. indistinguishable; K_{tool} against neither $p = 9.6 \times 10^{-8}$; K_{tool} against both $p = 0.0055$.

that survive the library half are not better reasoners but more stubborn: 228, 267 and 355 agent steps over 46 to 57 minutes, against fourteen steps and ten minutes for the median trial in that cell.

Two of the block’s seven sections cause this, and deleting them checks it. *Training discipline* offers a hundred epochs as a sane starting budget and tells the agent to raise it and train again rather than stop; *Time budget management* instructs it to assume it has “hours, not minutes” and warns that a training phase finishing “within a couple of minutes” signals undertraining. The untreated median trial here is two and a half minutes and is not undertrained: these are gradient-boosted trees on tabular data, and the block was written for convolutional and encoder models. Deleting those two sections and keeping the other five restores everything. Not one trial is lost of twenty-four, against thirteen with the full block ($p = 2.6 \times 10^{-5}$) and two with no block at all, a difference from the untreated cell that is not detectable ($p = 0.49$). The median trial returns to seven steps and three minutes, the bill to less than a sixth, and the whole sweep from one hour forty-six to twenty-six minutes.

On the library side the same assumption arrives as a fact rather than an instruction—the prescribed library is simply slow—which produces the sharpest cell in the design and its strangest pairing. The library half alone loses nineteen trials of twenty-two; adding the discipline half, which alone costs thirteen of twenty-four, brings the loss down to eleven of twenty-four ($p = 0.0055$). The instruction that harms by itself protects in combination, and the mechanism is legible: what the discipline half tells this harness is that a long-running job is normal, which is exactly what the library half requires it to believe.

The multi-agent system met none of this: the same four cells inside its own architecture lost respectively none, none, none and one trial of twenty-four (Section 7.1). Whatever absorbs the instruction there—a fixed workflow, a repair loop, a planner that owns the schedule—is what makes the prescription survivable, and none of it is part of the prescription. This is the sharper form of the paper’s thesis. Measured inside the system it ships with, the layer is worth +0.72% and costs nothing; the same bytes handed to a different architecture leave the answers untouched and cost most of them. A layer that has only ever been measured in one architecture has not been measured. The

practical consequence is smaller than that framing suggests: on the discipline side the text was two paragraphs away from being portable, and finding which two took one cell of twenty-four trials costing eleven cents. What does not port is not the knowledge but the assumption smuggled in beside it.

Two cautions.. Where the library cells do return a number it is better, +1.21% and +1.13% against the untreated cell, in the same direction and of the same order as the +0.64% the library half is worth inside the multi-agent system; but those figures average over the minority of trials that survived, and survival in those cells is exactly persistence, so we report the direction and suppress any stronger reading. Separately, several cells reproduce another system’s value to sixteen digits. That is not two architectures converging under a shared prescription: sberbank-housing returns 0.2516404543658208 in all three untreated multi-agent attempts and two of three untreated open-harness attempts, with no prescription anywhere. These tasks are deterministic given a library and its defaults, so any two runs reaching the same call reach the same number, and the prescription changes only which call that is; the treated cells agree with each other on 0.2514574079401016. Repeated attempts agreeing to every digit are therefore not independent samples of an agent, and the noise floors of Section 5 measure library determinism as much as agent stability.

7.5. What a different backbone changes

Table 8 varies the model: one smaller of the same generation and one stronger from another family, under both systems, with tasks, image, concurrency and machine unchanged.

Accuracy barely moves. Median change against each system’s own reference row is −0.06% and −0.54% for the open harness and −0.09% and −0.02% for the multi-agent system, against a bill varying elevenfold across the same rows. On cooking-time two different backbones under the open harness produced predictions identical to sixteen digits while deliberating differently: one tried XGBoost along the way, the other went straight to LightGBM, and both ended at the same call with the same defaults. The backbone changes the deliberation, not the artefact.

Table 8: Changing the backbone under each system on TabReD, with everything else held fixed. Deltas are against that system’s own `gemma-4-31b` row, signed so positive is better, and are computed only over trials that returned a metric. The two systems answer a stronger backbone in opposite directions: the open harness stops losing trials, the multi-agent system starts.

System	Backbone	Trials	Lost	Cut	In tok/trial	\$/sweep	Δ med.
<i>Terminus-2</i>							
	<code>gemma-4-26b-a4b</code>	24	6	–	720 720	0.926	-0.06%
	<code>gemma-4-31b</code>	24	2	–	37 317	0.069	ref.
	<code>glm-4.7</code>	24	0	–	110 602	0.761	-0.54%
<i>AutoDS-Tools</i>							
	<code>gemma-4-26b-a4b</code>	24	6	7	123 374	0.399	-0.09%
	<code>gemma-4-31b</code>	24	1	4	61 606	0.177	ref.
	<code>glm-4.7</code>	8	1	4	251 010	1.101	-0.02%

Lost: no metric returned. Cut: stopped at the task’s time limit, which does not always lose the trial. Timeouts were not recorded per trial for the open harness, whose trials finish far inside the limit.

On the normalised scale of Section 6 the whole backbone axis spans 0.24 under the open harness and 0.04 under the multi-agent system, against the 0.49 separating the three architectures at a fixed backbone once prescription is taken out, or 0.59 with the systems as they ship. The larger of the two is not a broad effect either: of that 0.24, 0.30 comes from `sberbank-housing` alone and the remaining seven tasks move the mean the other way by +0.07. On the architecture-only figure, architecture is the larger term by a factor of two against the worse-behaved backbone axis and twelve against the other.

Where accuracy does move it moves both ways, on the hinge this section is about: a stronger backbone exercises more discretion, and discretion departs from the gradient-boosting default. On `ecom-offers` that departure produced the best ROC-AUC anywhere in this paper, 0.627 against 0.554–0.569 for the reference model; on `sberbank-housing` the same departure to a random forest cost 46% of RMSE. Prescription and backbone are different levers on one hinge: whether the default being overridden was right to begin with.

Reliability separates the two systems, in opposite directions. Under the open harness, trials returning no metric fall from six of twenty-four to two to none as the backbone strengthens. Under the multi-agent system they are lowest at the model the system was built against and rise on both sides, from one of twenty-four to six below and one of eight above, with trials cut at the time limit moving the same way: seventeen per cent at the reference, twenty-nine below, fifty above. A stronger model does not make that architecture finish sooner; it gives the six agents more to say inside a fixed hour. When the binding constraint is the clock, a better backbone buys nothing the clock will let the system deliver.

Two cautions. The loss rate is partly a property of the deployment: the same reference model lost two trials of twenty-four here and none in the matched arm of Section 6.1, which ran on a different machine at half the concurrency, so only the within-deployment comparison is safe and that is what the table reports. And the axis has a floor we found by hitting it. Two models one generation older returned nothing usable: the smaller emitted tool calls with empty argument objects, the larger wrote a working pipeline but left its submission in the

wrong directory. Neither is a shortfall in accuracy; both are a failure to produce a measurement. Below some level of capability the question this axis asks stops having an answer, and that level is above what a parameter count would suggest.

7.6. Execution friction as a mediator

AutoDS-Tools contains a repair loop, invoked when execution fails. It is part of the system and active in both branches, but it does not fire equally often: across the 36 runs it was invoked 18 times with tool prescription against 12 without, in 9 of 18 runs against 8 of 18. The imbalance concentrates where the prescription pushes hardest—4 invocations against 0 on `clrs` and 3 against 0 on `amp-parkinsons`, both cases where the agent is steered to a specialised library whose API it then fails on—with the reverse occurring at smaller magnitude, 0 against 2 on `spaceship-titanic` and 3 against 4 on `imdb`.

Read as a mediator rather than noise, this quantifies the cost side. Prescription raises execution friction, and its net effect is positive only where the prescribed tool is enough better to repay it. It also means the effect we measure is that of prescription *as delivered by a system able to recover from the errors prescription causes*; in a harness without a repair loop the losses would plausibly be larger. This also explains the disagreement in the literature noted in Section 2: if “knowledge injection” bundles an override that helps with one that hurts, and the mix of task families differs between studies, aggregate ablations will report inconsistent and sometimes null effects for reasons unrelated to the systems compared.

What the losing tasks were told, and what they did.. The trial artefacts name the clause responsible. The discipline block instructs the agent not to stop after “an arbitrary small budget (5 or 15 epochs is almost always badly undertrained)” and offers “~ 100 epochs” as a starting point. On `clrs` the untreated branch chose 5, 10, 20 and 30 epochs over its run—exactly the range the clause forbids—and reached 0.3909 pointer accuracy, taking 92 minutes of a 120-minute budget on one attempt and running the budget out entirely on the other two while still submitting, at 0.4118 and 0.4404. The treated branch submitted nothing in three attempts. A branch that already consumes most

of the budget at 5 to 30 epochs cannot also honour a prescription calling those figures undertrained and naming a hundred. The instruction is not wrong as advice; it is written without reference to the budget the task grants, and on this task the two are incompatible. We cannot read the treated traces directly—the agent is killed at the ceiling before it writes one, itself a small piece of evidence—so what we report is the untreated configuration, the clause, and the outcome.

8. The starting point changes the sign

The two benchmarks differ in a way that is usually treated as a nuisance to be disclosed. TabReD supplies nothing and the task is solved from scratch; MAgentBench supplies a working training script and asks for improvement. We treat this as a factor rather than a caveat, because the value of prescriptive structure depends on it.

The win-loss counts split cleanly along that line. In the greenfield condition, the eight TabReD tasks where nothing is supplied, the branch carrying the prescription wins six of eight. In the improve-a-baseline condition, the MAgentBench tasks that each ship a working `train.py`, the branch without it wins six of ten when every cell is run once, and five of the eight we were able to run three times—with the difference that two of those five are now total, the prescribed branch submitting nothing at all in three attempts (Table 5). Both counts come from the same system, the same backbone and the same pair of instruction files; only the starting point differs. The reading is the one the mechanism predicts: where there is no default to override, any prescription that points the model at a reasonable tool improves on what it would otherwise construct, and where the supplied script already contains the decision the prescription was introduced to make, overriding it discards work that was correct.

The counts are symmetric but the magnitudes are not, and that asymmetry is the more informative half. Improving a baseline, the prescription can take a task from a usable score to no submission at all, three attempts of three (Table 5); on greenfield its normalised effect is smaller by an order of magnitude: a median of 0.014 across the eight tasks, bounded by `homecredit-default` at +0.118 and `ecom-offers` at −0.058, with no other task exceeding 0.06 in either direction. All eight effects still clear the run-to-run spread of Section 5, whose median on these tasks is 0.002, so they are real rather than noise—but on tabular data the prescription does very little work in either direction, for the reason Section 7.2 gives: the system reaches for the prescribed family unaided. Greenfield does not mean structure pays richly; it means structure stops costing. The TabReD half carries a caveat of its own, in that those eight runs used an earlier data preparation and are not leaderboard-comparable (Section 11), though they remain an internally valid contrast because both branches ran on the same preparation. The aggregate win-loss count across the two benchmarks should therefore not be read as a single comparison.

[TODO: this remains one factor observed across two protocols rather than a crossed design, and the completed K-disc grid does not fix it: the two benchmarks differ in

Table 9: Coverage and the reason for each absence.

System	TabReD	MAgentBench	Reason for absences
FEDOT.LLM	8/8	2/10	tabular inputs only, by design
AutoDS-Tools	8/8	10/10	—
Terminus-2	8/8	10/10	—

tasks, metrics and time budgets as well as in starting point. Closing it needs either a greenfield variant of the MAgentBench tasks or an improve-a-baseline variant of the TabReD ones, built so that only the starting point moves. Decide whether to build one or to state the confound and keep the weaker claim]

9. Limits of applicability

Coverage across the three systems is uneven, and the unevenness is a result rather than a gap. FEDOT.LLM attempts two of the ten MAgentBench tasks not because runs were abandoned but because its architecture accepts tabular inputs only; the segmentation, multi-label vision, graph and sequence tasks are outside what it can express, and would remain so after substantial engineering. Its domain is therefore the tabular core: the eight TabReD tasks plus `house-price` and `spaceship-titanic`.

We consequently report two things separately. Head-to-head comparisons are made on the tabular core, where all systems ran. Tasks outside it are reported as a coverage analysis, with an explicit reason for each absence, whether structurally unsupported, run failed, or not attempted, rather than as blank cells.

Terminus-2 used to be missing four MAgentBench tasks. The earlier job on this benchmark contains six trials, all completed with no errors, and `identify-contrails`, `feedback`, `clrs` and `fathomnet` were simply never submitted. Nothing structural prevented them, and we have since run all four: they are public, they need no accelerator, and they carry the same harness as the six. The row is now full, and the four added tasks behave no differently from the rest.

Closing that gap changed the row in a way we did not anticipate. Because the four tasks had to be re-run anyway, we re-ran the whole set on one job, one verifier and one concurrency setting, three attempts per task. Before, the row mixed six single trials from one job with four from another under different verifier glue, which is not a comparison so much as a concatenation. It is now homogeneous, and it is the first point in this paper at which two architectures are measured on MAgentBench under conditions that are identical rather than merely similar.

Read as a comparison, Table 10 says what the leaderboard of Section 6 already said, on a second benchmark and with the conditions held rather than assumed: the multi-agent system is the more accurate of the two, and the open harness is the cheaper. What it adds is the size of the second half. Terminus-2 takes about a third of the wall-clock per trial and reaches its agent budget once in 24 trials where AutoDS-Tools reaches it 13 times in 30; it also fails to submit more often, 6 of 24 against

Table 10: The two agentic architectures on MLAGentBench under identical conditions: one job, one verifier, one concurrency setting, three attempts per task, the same backbone. AutoDS-Tools is shown without the prescribed-knowledge layer, which is the branch that compares like for like with an open harness carrying no layer of its own. n counts the attempts of three that produced a submission, and brackets give their range; the mean is over those attempts. `cifar10` and `imdb` are omitted: neither repeats on hardware without an accelerator (Section 9).

Task	Metric	AutoDS-Tools	Terminus-2
<code>clrs</code>	Ptr. acc.	0.3909 $n=3$, [0.3204–0.4404]	0.1397 $n=2$, [0.1330–0.1464]
<code>ogbn-arxiv</code>	Accuracy	0.3866 $n=3$, [0.2237–0.4790]	0.1606 $n=3$, [0.0597–0.2655]
<code>spaceship-titanic</code>	Accuracy	0.8074 $n=3$, [0.8022–0.8131]	0.6590 $n=3$, [0.5859–0.7059]
<code>house-price</code>	MAE	15 894.8 $n=3$, [15 529.7–16 600.8]	21 594.8 $n=2$, [19 422.1–23 767.4]
<code>identify-contrails</code>	Dice	0.0022 $n=3$, [0.0000–0.0040]	0.0000 $n=2$, [0.0000–0.0000]
<code>amp-parkinsons</code>	SMAPE	62.2787 $n=2$, [58.2113–66.3461]	60.4999 $n=1$
<code>feedback</code>	MCRMSE	0.8261 $n=3$, [0.5403–1.2569]	0.5279 $n=2$, [0.4972–0.5586]
<code>fathomnet</code>	micro-F1	0.1435 $n=2$, [0.1428–0.1442]	0.2440 $n=3$, [0.0844–0.3846]

Counting a margin below 5% of the larger value as a tie, AutoDS-Tools takes 4 of the eight tasks and Terminus-2 2. Terminus-2 is nonetheless the faster of the two by a factor of about three—a median trial of 6.1 minutes against 10.9—and reached its agent budget once in 24 trials where AutoDS-Tools reached it 13 times in 30.

2 of 24 on these eight tasks. Neither of those is visible in a table of scores.

Two tasks we could not repeat.. Of the ten MLAGentBench tasks, eight were re-run three times in both branches for this paper. `cifar10` and `imdb` were not, and the reason is measured rather than assumed. Sixteen attempts out of sixteen exhausted the task’s own agent budget without writing a submission: six per branch at four concurrent trials, and four more at two, so contention is not the explanation. The agent does start—the analyst, planner and researcher reports appear in its working directory within the first minute—after which a single Python process holds about eight cores continuously. It is training, not looping. What separates these two from the rest is that they are the heaviest to train and are also given half the budget, 3600 s where five other tasks get 7200, while the task image installs the CUDA build of PyTorch by default and the task declares no accelerator, so the build falls back to the CPU on our hardware. The five-epoch starter script finishes on a CPU; nothing that improves on it does. These two rows keep their earlier single-trial figures and are marked as not repeated; the eight remaining tasks are reported at three attempts per cell. This is a limit of the hardware we had, not of the benchmark: on a machine with an accelerator both rows should repeat like the others.

Read this way, the coverage table is itself a finding: the architecture that performs worst on the tasks it can attempt is also the one that can attempt the fewest, and both properties follow from the same design decision.

10. Scientific case studies: the far end of the axis

Benchmarks measure what they were built to measure. To see the behaviour above outside benchmark conditions, we ran AutoDS-Tools on three published scientific studies, each with an author-reported baseline on the authors’ own split.

These runs are the opposite of a knowledge-free condition, which matters for reading them. Their instructions sit at the far end of the axis of Section 4: each names the library under a `REQUIRED` heading, supplies a working snippet that constructs it, identifies the target-leaking columns, describes the

split, and states the author baseline as the number to beat; the polymer instruction goes further and restates the published finding itself. What they measure is the ceiling under *maximal and correct* prescription—useful precisely because Section 7 shows prescription is not always correct.

Maize yield prediction.. Grain yield for field plots from the Genomes-to-Fields initiative, on the authors’ own split with no environment leakage. The agent reached Pearson $r = 0.569$ against the authors’ best model at $r = 0.461$ and their neural network at $r = 0.426$. The correlation is higher; the error is not. The agent’s normalised RMSE is 0.958 against the authors’ 0.948, and its R^2 on the held-out set is 0.083. It therefore ranks the plots better than the published model while predicting the yield itself no more accurately, a distinction that matters for the applied question and that a single-metric comparison would hide.

F-DATA exit-state prediction.. Job success or failure on Fugaku telemetry, using only attributes known at submission time, with the temporal split taken from the paper. The agent reached 92.07% accuracy against an author baseline of 89%. The instruction states the relevant context and the verifier confirms it: the classes are imbalanced and a constant “always success” predictor already scores 88.08%. Measured against that floor rather than against zero, the authors gain 0.9 points and the agent 4.0. Balanced accuracy is 0.705, so most of the remaining error is on the minority class, which is the class an operator would want predicted.

OpenPoly glass transition temperature.. Polymer property regression from PSMILES strings. The agent reached $R^2 = 0.543$ against an author baseline of 0.904, with MAE 40.4 K against 14.6 K. This comparison is *not* like-for-like: the authors used the full dataset, we were unable to obtain it, and the agent worked from a subset whose test split is 89 rows. We report the row for completeness and draw no conclusion from it.

What this does and does not show.. Two of three author baselines were exceeded on the metric the authors reported, by a

31B open model. That is a real result about applied scientific baselines, which are frequently well below what a competent tabular pipeline achieves, and it supports reading the ceiling of Section 6 as a statement about the gap to *tuned* tabular baselines specifically rather than about the systems being weak in general. It is not a result about agents working unaided: the knowledge that would otherwise have to be discovered, which library, which featurisation and which columns leak, was supplied in the prompt. Read against Section 7 the two findings agree. Prescription pays where the model’s own default would be wrong, and on an unfamiliar modality such as polymer SMILES the default would certainly be wrong; the tabular benchmark tasks, where the default is already right, are where the same prescription buys nothing.

These are three single points and not a design. Running the other two systems on the same cases, once with these instructions and once stripped to the task statement, would make it a two-by-three crossing and would test Section 7.4 on the modality where prescription actually pays. We did not: unlike the benchmark tasks, these exist only as the runs that produced them, with no task definitions, splits or scorers in the environment we rebuilt.

11. Threats to validity

Comparability of data slices and provisioning.. Table 2 is restricted to systems run through the adapter that uses the full official temporal splits and the benchmark’s own metrics in raw target units. Earlier AutoDS-Tools runs used a different preparation: a 40,000-row training slice spanning the official train and validation splits, a test slice covering 16.7%–61.4% of the official test window in every case its final rows, and an extra row-order column the official protocol withholds. Measured split sizes for all eight tasks are released; those runs are excluded here and the leaderboard uses a re-run of the same system on the common adapter, so the difference between the two sets can be read as the price of the preparation. The earlier comparison also differed in what was installed, the AutoDS-Tools arm carrying the libraries its prescription names while the others used the stock image. Section 6.1 closes that axis directly—offered the enriched image, the open harness scores inside its own noise floor on seven of eight tasks and never names a library the image adds—so the threat stands only for the FEDOT.LLM runs, which we could not repeat.

Our own provisioning was broken, and the systems repaired it.. The enriched image brings LightGBM as a dependency, but the base image ships no OpenMP runtime and the wheel carries none. The import succeeds and the failure arrives inside the fit, as a broken worker pool that reads as a modelling error and is not one. Forty-five trials met it—twenty-two of twenty-four AutoDS-Tools trials and twenty-three of twenty-four matched-provisioning trials—and all forty-five diagnosed and repaired it in a single step, which is why it went unnoticed until we read the traces. Repeating the whole arm on a corrected image agrees to within 0.13% on seven tasks and 0.34% on the eighth, so the defect cost no measurable accuracy. It could still have distorted

a crossed design, since the fault sits inside the prescribed library and can strike only the cells where that prescription is on. Every cell we report ran on the corrected image.

Repeats and numeric precision.. Published baselines are means over fifteen seeds; our agent rows are one run for FEDOT.LLM and the mean of three attempts otherwise. The two agentic systems do not share a noise floor: Terminus-2 reaches a relative standard deviation of 0.94% while AutoDS-Tools reproduces itself to within 0.07%, so the 1% threshold used across systems is set by the noisier of the pair and would be far too coarse for comparing configurations of one system, where a tenth of a percent is already outside the spread. FEDOT.LLM metrics reached us at two decimal places. Sweeping every task independently to both ends of its rounding interval moves its average rank between 16.6 and 20.3 of 21 and its normalised mean between 0.05 and 0.47; no claim here depends on any part of that interval, but its individual rank cells should be read as indicative.

Scorers that accept degenerate submissions.. The prohibition on constant submissions belongs to the prescribed-knowledge layer, not to any task: fingerprinting the layer shipped with each of the ten MLAGentBench tasks shows it present in all ten, byte-identical within a family. It therefore reached the treated branch of AutoDS-Tools everywhere and reached neither the untreated branch nor Terminus-2, whose harness receives the task file alone. The asymmetry is between arms rather than tasks and, since the clause constrains rather than assists, it runs against the only arm that carried it. Both degenerate submissions we found belong to Terminus-2. On imdb it submitted one label for all 25,000 test rows and the scorer returned an accuracy of exactly 0.5000, recording a submission with no model in it at chance level rather than rejecting it; on amp-parkinsons 344 of 559 rows (61.5%) carry the same global value in all four targets. We exclude the imdb figure and read amp-parkinsons as a partial fallback. The wider point belongs to the benchmark: on eight of the ten tasks nothing in the scoring path separates a trained model from a constant, so an agent that fails silently is scored as though it had merely performed poorly.

An active repair loop inside the treated system.. The repair loop of AutoDS-Tools is architecture rather than prescription, but it is invoked asymmetrically between the branches (Section 7) and therefore partially masks the cost of prescription. We report invocation counts alongside the scores rather than adjusting for them. Disabling it in both branches would measure the interaction between prescription and repair, a different quantity which we did not run and which is the experiment we would run next: the repair loop is the most plausible explanation for why prescription is survivable in this architecture and not in the other (Section 7.4).

Trials lost to the wall-clock ceiling.. Two of the twenty-four AutoDS-Tools trials on the common adapter reached the 3600 s limit before writing a submission and are excluded, so `homecredit-default` and `maps-routing` rest on two attempts. Scoring them as zero

was considered and rejected: a timeout measures the budget, not the model, and averaging one in would move `homecredit-default` from 0.863 to 0.575 and drop the system from third place to near-last—an artefact that would have looked entirely plausible. The exclusion is not neutral either, since it drops the slowest attempts and leaves the means mildly optimistic about a system whose failures correlate with the tasks it finds hardest.

Data and validation access. Several MAgentBench tasks use subsets of the original data and none uses an official competition test split, so absolute values there are not comparable with public leaderboards and every comparison in this paper is internal by construction. On TabReD the agents receive a labelled validation split whereas the published baselines train on the training split alone—a small advantage to the agents, working against the direction of our main conclusion.

The prescription is a moving target. The layer is composed on the host by the agent package installed when the runner was set up, not by the copy pip places inside the container, so reinstalling from a newer checkout changes the treatment without changing anything visible in the run configuration. We fixed the version by measurement rather than by tag, and the two benchmarks turn out to have run against two editions. Every TabReD run applied the layer composed by the branch we cloned: 4972 characters for tabular tasks, SHA-256 prefix `a58b4d40678ff1aa`, verified byte-identical to the tail of `agent/instruction.md` in each trial. The MAgentBench runs applied the edition shipped with the tasks: 5085 characters for tabular (`bf1e4e7c8c4960a2`), 4363 for vision, 4163 for text, 4153 for graph. The two carry the same eight section headings and differ by 113 characters in the shared discipline block and 668 in the graph library block. The difference is not cosmetic: the cloned edition advises that SMAPE penalises a non-zero prediction against a zero target maximally and tells graph agents to mask unreachable candidates before the softmax—two hints written for particular tasks—where the shipped edition carries the general prohibition on constant submissions instead. The repository recomputes every fingerprint above.

We ran the two editions against each other rather than leaving the threat at a character count: same system, same eight tasks, same images, verifier and concurrency, three attempts each. Under the shipped edition the treated branch produces no submission at all on `feedback`, `clrs` and `fathomnet`, nine attempts of nine; under the edition composed today it submits on all three, at MCRMSE 0.5512, pointer accuracy 0.1281 and micro-F1 0.1507. Where the budget was never binding the shipped edition is the better of the two—SMAPE 86.46 against 93.15, accuracy 0.5069 against 0.4683—and the remaining pair is level. Counting submissions rather than scores, the shipped edition yields 15 of 24 and the composed edition 21 of 24. The edit is thus not an improvement or a regression but a move along the axis this paper is about: one edition presses harder on training discipline and pays in unfinished runs, the other presses less and finishes. On three of eight tasks the gap between two editions of one prescription exceeds the gap between having the prescription and not having it. A result of this kind cannot be

repeated from a description of the layer, only from its bytes, and the bytes are what is least often reported.

Single backbone family. The comparison uses one model throughout. Section 7.5 varies it under both systems and finds the accuracy ordering unchanged, so the ranking is not a backbone artefact; what the backbone changes is reliability and cost, and in opposite directions for the two architectures. That rests on three models, two from one vendor, and one task family. A stronger backbone that reasons less verbosely might not push the multi-agent system past the time limit at all, and we cannot separate capability from verbosity with what we ran.

12. Conclusion

We isolated the architectural axis of agentic AutoML by holding the backbone model fixed and varying only the surrounding system, and we measured the result against reference methods that were tuned.

Architecture is worth a great deal and not quite enough. Under an identical open 31B model, the systems span 0.26 to 0.84 of the range defined by eighteen expert-tuned baselines, while tuned XGBoost sits at 0.87; the strongest of them is level with tuned LightGBM and ahead of tuned CatBoost. The gap architecture closes is real and large; the one it leaves open has narrowed to 0.02 but has not closed.

The relationship is not monotonic in architectural weight, in either direction. The most constrained system ranks near the bottom and the highest-ranking of the three is neither extreme, which suggests the useful question is not how much structure a system imposes but which decision that structure removes. Our evidence is that constraint pays where it overrides a default the weak model gets wrong and costs where the default was already right, visible in an ablation whose sign flips between task families. The two directions are not balanced: at three attempts per cell the costly one is total on three tasks of eight while the paying one rests on a single task. What we can assert is the cost of override; the benefit is the weaker half of the claim, and one that plausibly explains why aggregate knowledge and memory ablations in the literature disagree.

For practitioners the implication is narrow. On established families with strong classical solutions a tuned baseline remains the correct choice, and an agentic system’s value lies in coverage, cost, and reaching tasks where no such baseline exists—as the case studies show, where two of three published author baselines were exceeded by an open 31B model told which library and featurisation to use. What the practitioner supplies, and whether the system would have found it alone, is the variable to watch.

Data availability

All tasks, both instruction variants and every execution trace behind these numbers are public. The benchmark adaptations and the two instruction files per task are on HuggingFace as `danil-e/harbor-datasets-tabred` and

danil-e/harbor-datasets-mlab; the tabular adapter is AaLexUser/harbor-tabred-adapter. Trajectories, submissions, scorer outputs and per-job telemetry for the earlier runs are on the Harbor hub under job identifiers 7593cc7b and 1a42d1fe (Terminus-2 on TabReD and on MLAGent-Bench) and 3113aba7 and 56768b37 (the two AutoDS-Tools branches); any trial can be retrieved with harbor hub trial download, and the values in our tables were checked against those files task by task. The repeats reported here ran on our own hardware and are released with the data package under the job names autods-mlab-nolayer-full, autods-mlab-c1-full, autods-mlab-layer-now and terminus-mlab-full. The package also carries the consolidated metric table across all systems and benchmarks, the measured official split sizes for the eight TabReD tasks, the per-task tables this paper reports in summary, the learner and hyperparameter extraction of Section 7.2, the job telemetry of Table 4, the fingerprints of both editions of the prescribed-knowledge layer with the script that recomputes them, and the scripts that turn all of it into every table printed here.

[TODO: repository DOI once deposited]

Acknowledgements

[TODO: funding, compute, and people to thank; empty at submission is a defect, not a placeholder]

References

- [1] I. Rubachev, N. Kartashev, Y. Gorishniy, A. Babenko, TabReD: Analyzing pitfalls and filling the gaps in tabular deep learning benchmarks (2024). arXiv:2406.19380. URL <https://arxiv.org/abs/2406.19380>
- [2] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al., Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, in: International Conference on Learning Representations (ICLR), 2026, terminal-Bench 2.0; introduces the Terminus 2 scaffold and the Harbor harness. arXiv:2601.11868. URL <https://arxiv.org/abs/2601.11868>
- [3] Y. Yao, X. Tan, C.-H. Liu, Y. Li, Z. Wang, W. Yu, Z. Tan, Y. Tian, G. Zhao, L. Sun, X. Zhang, T. Yang, Harness-Bench: Measuring harness effects across models in realistic agent workflows (2026). arXiv:2605.27922. URL <https://arxiv.org/abs/2605.27922>
- [4] N. Vats, O. Golev, The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation (2026). arXiv:2607.22585. URL <https://arxiv.org/abs/2607.22585>
- [5] S. Kulibaba, A. Dzhalilov, R. Pakhomov, O. Svidchenko, A. Gasnikov, A. Shpilman, KompeteAI: Accelerated autonomous multi-agent system for end-to-end pipeline generation for machine learning problems (2025). arXiv:2508.10177. URL <https://arxiv.org/abs/2508.10177>
- [6] H. Fang, et al., MLZero: A multi-agent system for end-to-end machine learning automation (2025). arXiv:2505.13941. URL <https://arxiv.org/abs/2505.13941>
- [7] Z. Li, Q. Zang, D. Ma, J. Guo, T. Zheng, M. Liu, X. Niu, Y. Wang, J. Yang, J. Liu, W. Zhong, W. Zhou, W. Huang, G. Zhang, AutoKaggle: A multi-agent framework for autonomous data science competitions (2024). arXiv:2410.20424. URL <https://arxiv.org/abs/2410.20424>
- [8] P. Trirat, W. Jeong, S. J. Hwang, AutoML-Agent: A multi-agent LLM framework for full-pipeline AutoML (2025). arXiv:2410.02958. URL <https://arxiv.org/abs/2410.02958>
- [9] E. Toledo, K. Hambardzumyan, M. Josifoski, R. Hazra, N. Baldwin, A. Audran-Reiss, M. Kuchnik, D. Magka, M. Jiang, A. M. Lupidi, A. Lupu, R. Raileanu, K. Niu, T. Shavrina, J.-C. Gagnon-Audet, M. Shvartsman, S. Sodhani, A. H. Miller, A. Charnalia, D. Dunfield, C.-J. Wu, P. Stenertorp, N. Cancedda, J. N. Foerster, Y. Bachrach, AI research agents for machine learning: Search, exploration, and generalization in MLE-bench (2025). arXiv:2507.02554. URL <https://arxiv.org/abs/2507.02554>
- [10] A. Tsymbalov, D. Zaripov, A. Epifanov, A. Palienko, GRACE-DS: a guarded reward-guided agent correction environment in data science, normalised scale anchored on an untuned HistGradientBoosting oracle; eight frontier/MoE backbones, no dense small model (2026). arXiv:2606.16000. URL <https://arxiv.org/abs/2606.16000>
- [11] J. Chen, J. Shen, H. Kang, Z. Hong, Q. Jiang, S. Bose, Y. Zhang, L. Leng, A. Vyas, L. Mao, S. Ouyang, K. Zhou, L. Qin, AgentSpec: Understanding embodied agent scaffolds through controlled composition (2026). arXiv:2606.14674. URL <https://arxiv.org/abs/2606.14674>
- [12] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, Y.-G. Jiang, Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses (2026). arXiv:2604.25850. URL <https://arxiv.org/abs/2604.25850>
- [13] N. O. Nikitin, P. Vychuzhanin, M. Sarafanov, I. S. Polonskaia, I. Revin, I. V. Barabanova, G. Maximov, A. V. Kalyuzhnaya, A. Boukhanovsky, Automated evolutionary approach for the design of composite machine learning pipelines, Future Generation Computer Systems 127 (2022) 109–125. doi:10.1016/j.future.2021.08.022.

- [14] Q. Huang, J. Vora, P. Liang, J. Leskovec, MAgentBench: Evaluating language agents on machine learning experimentation (2024). `arXiv:2310.03302`.
URL <https://arxiv.org/abs/2310.03302>